

FA Support Agent — Full workflow

Telegram customer-support agent for Fantasy Adda on a personal account (GramJS / MTPROTO). Repository: github.com/adityakushwaha8421-crypto/AddaAISupport · Version documented: 25 Sep 2026, greeting rule included.

1. What the agent does

The agent sends exactly three kinds of message to customers, and nothing else:

- One evidence request per case.** When a customer's message is clearly a **deposit** issue (money paid into the wallet, not showing) or a **withdrawal** issue (money withdrawn, not in the bank), it sends the list of documents the team needs, once, in the customer's language. After that it is completely silent in that chat: no reminders, follow-ups, acknowledgements or status updates. The human team handles everything.
- One solved note per confirmed payment.** When the export bot sends  **PAYMENT CONFIRMED** naming a `User ID`, exactly that customer is told once: *"Sir, aapka issue solved ho gaya hai. Sorry for the inconvenience. 🙏"* (English and Hindi versions exist).
- One greeting per fresh conversation.** A message that is only a greeting ("Hi", "Hello", "Hlo", "Namaste", "Good morning sir") is answered with one greeting, in the customer's language, **only** when it opens a conversation: no open case in the chat, nothing else said in the chat either way in the last 48 h, and no greeting already answered in that time (kept on the customer's record, so a restart never greets twice). A "hi" inside a case, after a solved case or after an unanswered message gets nothing. "Hi deposit nahi hua" is not a greeting: it is classified like any other message.

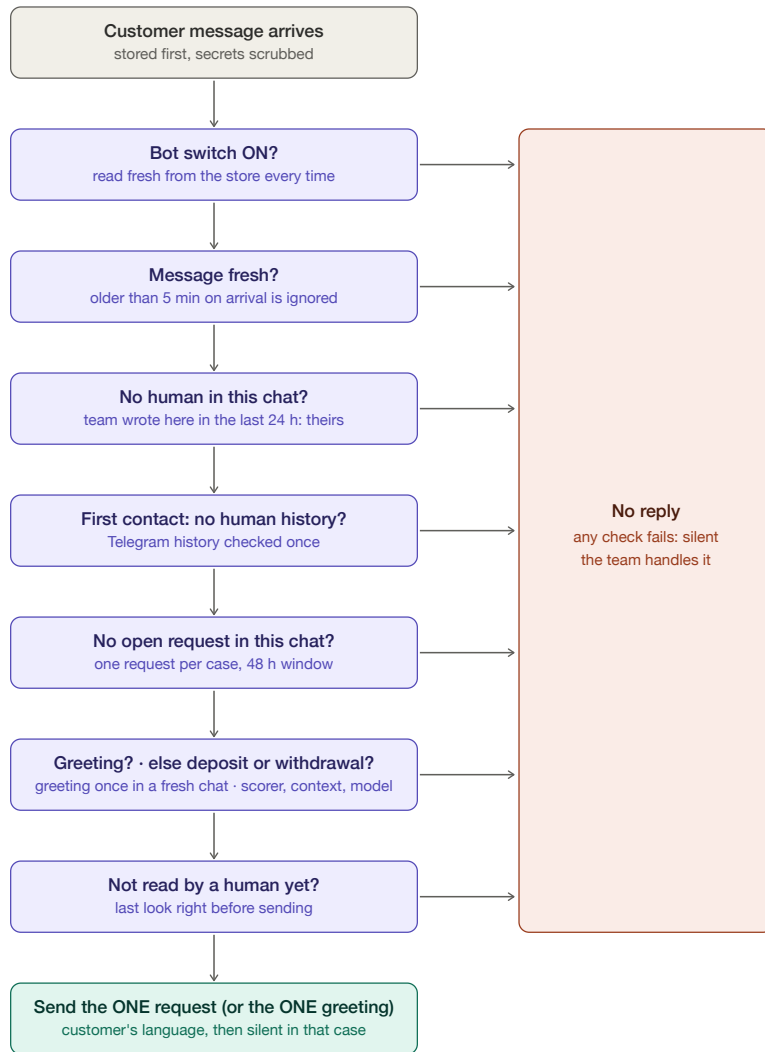
Everything else — thanks, match issues, app or login problems, unclear money messages, bare photos — is received and stored and gets no reply. There is no clarifying question and no guess.

What the request asks for

Case	Requested, once
Deposit	10-digit registered number · payment screenshot · bank statement PDF of the account paid from · screen recording (video) of the payment
Withdrawal	Withdrawal ID or withdrawal-history screenshot · bank statement PDF of the account the amount should have reached

2. Customer message flow

Every message is stored first, then passes these checks in order. Any failed check means silence; the team handles it.

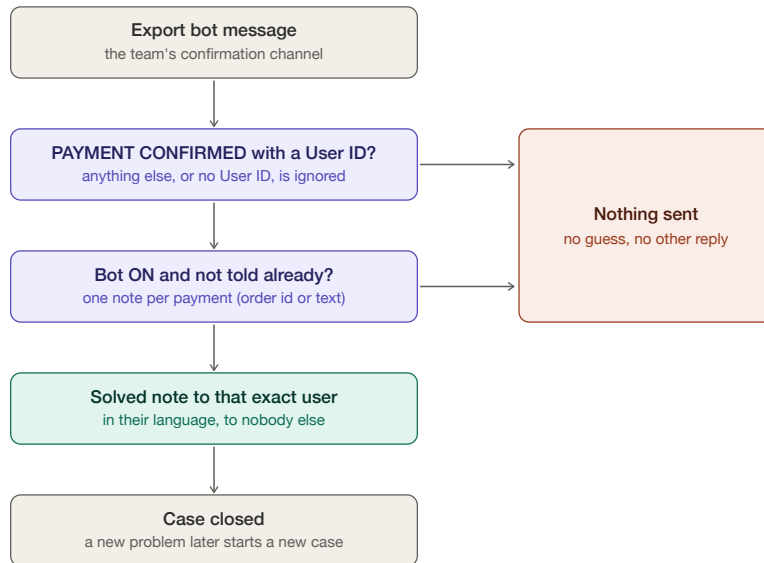


The checks in detail

Check	Rule	Setting
Bot switch	<code>/botoff</code> sets <code>bot_enabled=false</code> in the store and in <code>data/bot-state.json</code> ; survives restart, <code>/restart</code> and reconnects. Checked first, and again inside the transport right before every send.	—
Freshness	A message older than 5 minutes when handled (restart, reconnect catch-up) is never answered; old messages are never replayed after a restart.	<code>STALE_MESSAGE_SECONDS=300</code>
Human takeover	Any message the account sends in a customer chat marks that chat as a human's for 24 h from the human's latest message.	<code>HUMAN_TAKEOVER_HOURS=24</code>
First contact	The chat's Telegram history is read once; a message from the account that the agent did not send means a human is already talking to the customer. Unreadable history → silent for that message, re-checked next time.	—
Open request	One request per case. While a request younger than 48 h exists in the chat, nothing is sent, whatever the customer writes, including a different kind of problem.	<code>CASE_REOPEN_HOURS=48</code>
Greeting	“Hi” / “Hello” / “Hlo” / “Namaste” and nothing else. Answered once, only when it opens a conversation: no open case, nothing but greetings said in the chat within 48 h, no greeting answered within 48 h. Otherwise silent, like everything else in a case.	<code>CASE_REOPEN_HOURS=48</code>
Issue detection	Match cues first (a match problem is never a payment case). Then the direction-of-money scorer (Hinglish/Hindi/English, misspellings). A vague follow-up borrows the direction from the customer's recent messages. Only if nothing is decisive is the model asked once, with that history as context. Not clearly deposit or withdrawal → nothing.	<code>OPENAI_API_KEY</code>
Read by a human	A message already read on Telegram by a person is theirs to answer.	<code>REPLY_ONLY_TO_UNREAD=true</code>

3. Payment confirmation flow

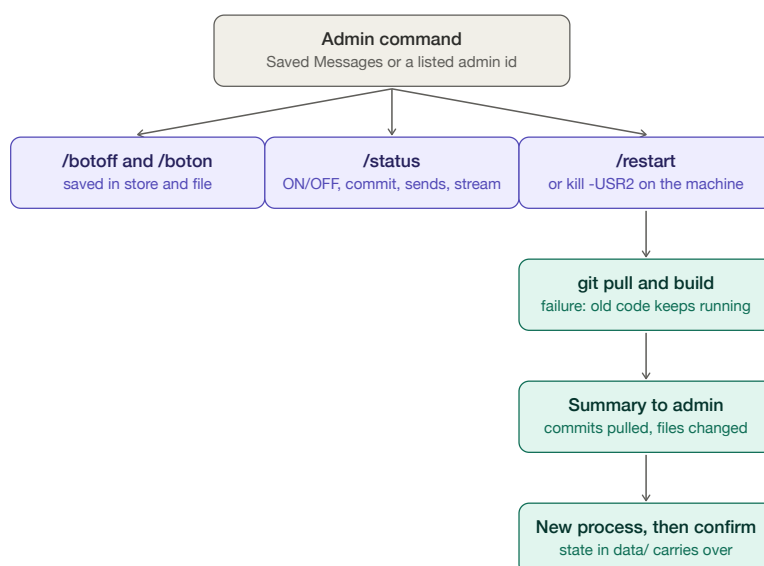
The only other customer-facing message. It is triggered by the team through the export bot, never by a customer message.



- A valid confirmation contains `PAYMENT CONFIRMED` and a `User ID: <telegram id>` line. The customer is never guessed from a mobile number or a reply.
- Duplicates are prevented per payment: by order id when present, otherwise by the confirmation text. The same order confirmed twice sends nothing more.
- The note is sent even when no stored case exists for that user (for example after a restart). If a case exists, it is closed.
- Messages such as "Files received", "MANUAL REVIEW NEEDED" or "PAYMENT PENDING" send nothing.

4. Admin commands and restart

Accepted from the account owner in **Saved Messages**, and from any user id listed in `ADMIN_TELEGRAM_IDS`. From anyone else the same words are ordinary customer text and do nothing.



Command	Effect	Reply
/botoff	Bot OFF. Saved in the store and <code>data/bot-state.json</code> . Nothing automatic goes out; messages arriving while OFF are never answered later.	🚫 Bot is OFF

/boton	Bot ON. Only messages arriving after this are handled.	✅ Bot is ON
/status	ON/OFF, running commit and uptime, requests and solved notes sent since start, Telegram stream take-overs, old-bot replies seen in 24 h.	status text
/restart	<code>git pull --ff-only</code> , <code>npm install</code> if the lockfile changed, <code>npm run build</code> , then a fresh process on the new code. A failed pull or build leaves the old process running.	🔄 Pulling... → 📄 summary → ✅ Bot restarted successfully. Running <commit>

5. Safety nets

- **Guarded transport.** Every automatic send goes through a wrapper that refuses any message to a customer chat unless its kind is on the allowlist (`evidence_request` , `payment_confirmed`), and refuses everything while the switch is OFF, re-reading the switch immediately before the send. Admin replies use the raw transport so they work while OFF.
- **Update-stream watchdog.** Telegram gives a session's updates to its newest connection. If another connection ever takes the stream, the agent detects that Telegram's counter moved while nothing arrived and reconnects. The session string must exist in exactly one place.
- **Old-copy detection.** Messages the account sends that this process did not send are matched against the old bot's wording; matches are logged and shown in `/status` .
- **State on disk.** With `STORE=memory` , the ON/OFF switch, the request ledger and customer state (language, takeover, conversation check) live in `data/` , so a restart never causes a second request or forgets a takeover.
- **One instance per account.** A PID lock (`secrets/agent.lock`) prevents two copies on the same machine.
- **Secrets.** Session strings, API keys and passwords are never logged; the logger scrubs every configured secret. `.env` , `secrets/` , `data/` and `logs/` are never committed.

6. What is stored

Table / file	Contents
<code>users</code> (<code>data/users.json</code>)	Telegram id, chat id, username, first name, detected language, human takeover until, conversation-check time
<code>messages</code> (memory only)	Every inbound and outbound message: text after scrubbing, media references, reply-to id, Telegram date
<code>evidence_requests</code> (<code>data/requests.json</code>)	One row per case: chat, user, issue type, language, status (sending → sent → solved), the request's Telegram id
<code>settings</code> (<code>data/bot-state.json</code>)	<code>bot.enabled</code> , <code>bot.enabledAt</code> , one key per payment already told

7. Running it

```
cp .env.example .env      # fill in TELEGRAM_API_ID, TELEGRAM_API_HASH, OPENAI_API_KEY, EXPORT_BOT_ID
npm run telegram:session   # one-time login: writes TELEGRAM_SESSION into .env (keep private, one machine only)
npm run build && npm start  # run
npm test                  # 168 offline tests; RUN_LLM_EVALS=1 npm test -- tests/evals for the live model accuracy run
```

Machine-side: `kill -INT <pid>` stops cleanly; `kill -USR2 <pid>` is the same update-and-restart as `/restart` .